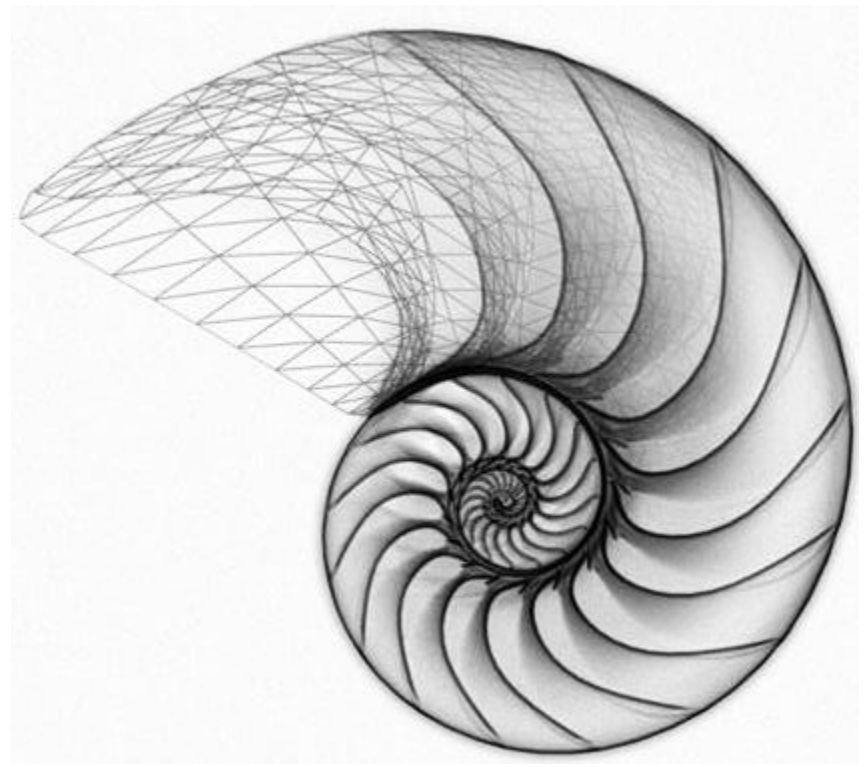




# Lecture 04

## C++ Primer

CSE225: Data Structures and Algorithms

# A Complex Number Example

```
class Complex
{
    public:
        Complex();
        Complex(double, double);
        Complex Add(Complex);
        void Print();
    private:
        double Real, Imaginary;
};
```

```
Complex::Complex()
{
    Real = 0;
    Imaginary = 0;
}
```

```
Complex::Complex(double r, double i)
{
    Real = r;
    Imaginary = i;
}
```

```
Complex Complex::Add(Complex c)
{
    Complex t;
    t.Real = Real + c.Real;
    t.Imaginary = Imaginary + c.Imaginary;
    return t;
}

void Complex::Print()
{
    cout << Real << endl;
    cout << Imaginary << endl;
}
```

```
int main(void)
{
    Complex A(1,1), B(2,3);
    Complex C;
    C = A.Add(B);
    C.Print();
    return 0;
}
```

# Friend Function

- The function has access to the private members in the class declaration
- The function is not in the scope of the class

# Friend Function

```
class Complex
{
    public:
        Complex();
        Complex(double, double);
        void Print();
        friend Complex AddComplex(Complex, Complex);
    private:
        double Real, Imaginary;
};
```

```
Complex::Complex()
{
    Real = 0;
    Imaginary = 0;
}
```

```
Complex::Complex(double r, double i)
{
    Real = r;
    Imaginary = i;
}
```

# Friend Function

```
void Complex::Print()
{
    cout << Real << endl;
    cout << Imaginary << endl;
}

Complex AddComplex(Complex a, Complex b)
{
    Complex t;
    t.Real = a.Real + b.Real;
    t.Imaginary = a.Imaginary + b.Imaginary;
    return t;
}

int main(void)
{
    Complex A(1,1), B(2,3);
    Complex C;
    C = AddComplex(A, B);
    C.Print();
    return 0;
}
```

# Operator Overloading

- We can use some operator symbols to define special member functions of a class
- Provides convenient notations for object behaviors

# Operator Overloading

```
class Complex
{
    public:
        Complex();
        Complex(double, double);
        Complex operator+(Complex);
        void Print();
    private:
        double Real, Imaginary;
};
```

```
Complex::Complex()
{
    Real = 0;
    Imaginary = 0;
}
```

```
Complex::Complex(double r, double i)
{
    Real = r;
    Imaginary = i;
}
```

# Operator Overloading

```
Complex Complex::operator+(Complex a)
{
    Complex t;
    t.Real = Real + a.Real;
    t.Imaginary = Imaginary + a.Imaginary;
    return t;
}

void Complex::Print()
{
    cout << Real << endl;
    cout << Imaginary << endl;
}

int main(void)
{
    Complex A(1,1), B(2,3);
    Complex C;
    C = A + B;
    C.Print();
    return 0;
}
```